

IncidentIQ

Seven-Person User Stories and Sprint Execution Plan

Detailed owner-by-owner work breakdown for completing the hackathon submission by tomorrow.

Team Plan Overview

Owner	Primary Deliverable	Integration Point
Person 1	n8n workflow	Calls FastAPI and routes Slack/JIRA branches.
Person 2	Gradio dashboard	Calls FastAPI or pipeline runner.
Person 3	FastAPI backend	Shared API for Gradio and n8n.
Person 4	RAG engine	Supplies context to RCA/remediation agents.
Person 5	Parser/classifier/timeline agents	Feeds structured evidence into graph.
Person 6	RCA/remediation/action agents	Produces final incident intelligence.
Person 7	Integration, QA, deployment, README, demo	Keeps submission coherent and hosted.

Parallel Work Rule

Everyone should code against the shared response contract. If a dependency is not ready, use a stub returning the same shape. This avoids blocking the team while preserving integration safety.

Person 1 - n8n Workflow Owner

Automated workflow routing and integration orchestration

As a DevOps/SRE user, I want n8n to receive incidents and route critical issues to Slack/JIRA so that the demo proves real workflow automation.

Owned Files / Artifacts

- workflows/n8n_incidentiq_workflow.json
- docs/n8n_setup.md

Detailed Steps

- 1 Create a Webhook Trigger that accepts a log analysis request or a completed analysis payload.
- 2 Add an HTTP Request node that calls FastAPI POST /analyze with sample logs or a staged file reference.
- 3 Add a Switch node using highest_severity with branches for P1/P2, P3, and Unknown.
- 4 Route P1/P2 to Slack and JIRA nodes; route P3 to Slack/report only; route Unknown to report-only.
- 5 Add HTTP Response node returning run_id, status, highest_severity, report links, and payload previews.
- 6 Export the workflow JSON and add a short import/run guide.
- 7 Capture screenshots of successful n8n execution for README/demo.

Acceptance Criteria

- n8n can call the hosted or local FastAPI /analyze endpoint.
- P1/P2 route reaches Slack/JIRA branch; P3/Unknown does not create JIRA.
- Workflow JSON is committed and documented.
- Demo can show n8n execution history in under 45 seconds.

Dependencies

- FastAPI /analyze contract
- sample payloads
- optional Slack/JIRA credentials

Person 2 - Gradio Frontend Owner

Interactive incident dashboard

As a judge, I want a public dashboard where I can upload logs and inspect the final analysis without installing anything.

Owned Files / Artifacts

- src/ui/gradio_app.py
- app.py

Detailed Steps

- 1 Build Gradio Blocks UI with file upload, run button, status trace, and result tabs.

- 2 Support .zip, .jsonl, .log, and .txt upload.
- 3 Call FastAPI /analyze or a shared local pipeline function depending on deployment mode.
- 4 Render incident table with severity, category, affected service, confidence, and evidence count.
- 5 Render detail tabs for timeline, RCA, remediation, cookbook, Slack preview, JIRA preview, and raw JSON.
- 6 Add download buttons for Markdown and JSON report artifacts.
- 7 Make the UI usable without API keys by displaying deterministic fallback results.

Acceptance Criteria

- Public URL opens and accepts sample logs.
- The dashboard shows at least one complete incident from provided datasets.
- Slack/JIRA previews are visible in the UI.
- Judges can download or copy final report JSON/Markdown.

Dependencies

- FastAPI endpoint or pipeline runner
- shared response schema
- sample datasets

Person 3 - FastAPI Backend Owner

Stable backend API for Gradio and n8n

As frontend and automation clients, Gradio and n8n need a reliable HTTP API that accepts logs and returns standardized incident intelligence.

Owned Files / Artifacts

- src/api/main.py
- src/api/schemas.py
- Dockerfile
- .env.example

Detailed Steps

- 1 Create FastAPI app with GET /health, POST /analyze, GET /runs/{run_id}, and GET /reports/{run_id}.
- 2 Define Pydantic request and response models matching the shared output contract.
- 3 Accept multipart file uploads and JSON requests with staged file paths.
- 4 Call the LangGraph/pipeline runner and persist outputs under outputs/{run_id}.
- 5 Return consistent JSON even if LLM, RAG, Slack, or JIRA is unavailable.
- 6 Enable CORS for Gradio and n8n.
- 7 Add Docker or Hugging Face/Render entrypoint.

Acceptance Criteria

- GET /health returns ok.

- POST /analyze works with block1 sample data.
- Gradio and n8n can consume the same response shape.
- No API key is required for deterministic demo mode.

Dependencies

- pipeline runner
- models
- output contract

Person 4 - RAG Owner

Runbook/SOP retrieval for grounded remediation

As a remediation agent, I need relevant SOP and prior-incident context so that recommended actions look credible and traceable.

Owned Files / Artifacts

- runbooks/*.md
- src/rag/runbook_store.py
- src/rag/retriever.py
- src/rag/vector_store.py

Detailed Steps

- 1 Create mock SOP markdown files for database, timeout, auth, memory, queue, disk, dependency, and deployment incidents.
- 2 Implement a RunbookStore that loads and chunks local markdown documents.
- 3 Implement lexical retrieval using category, service, error terms, and signal keywords.
- 4 Return top three chunks with title, source, score, and excerpt.
- 5 Add optional OpenAI embedding path if OPENAI_API_KEY is present.
- 6 Add optional FAISS/Weaviate adapter only after lexical fallback is stable.
- 7 Expose retrieved context to RCA and remediation agents.

Acceptance Criteria

- Known incident categories retrieve relevant SOP content.
- Pipeline still works if embeddings/vector DB are unavailable.
- Final output lists retrieved runbook context.
- README explains current fallback and target Weaviate architecture.

Dependencies

- incident category/service/evidence fields
- mock SOP content

Person 5 - Agent Owner A

Ingestion, signal extraction, classification, and timeline

As the analysis system, I need to convert noisy logs into structured evidence and incident candidates for downstream reasoning.

Owned Files / Artifacts

- src/models/*.py
- src/ingest/*.py
- src/signals/*.py
- src/clustering/*.py
- src/agents/classifier_agent.py
- src/agents/timeline_agent.py

Detailed Steps

- 1 Extract v4 notebook models for LogEvent, SignalEvidence, EvidenceCluster, IncidentAnalysisResult, and TimelineEvent.
- 2 Implement safe file loading, ZIP extraction, log discovery, parsing, and normalization.
- 3 Implement deterministic regex/signal rules for database, network, auth, memory, timeout, queue, disk, deployment, and dependency failures.
- 4 Implement clustering by file, category, service, time bucket, and signature where available.
- 5 Implement classifier agent with severity/category fallback heuristics.
- 6 Implement timeline agent by sorting cited evidence and selecting representative events.
- 7 Ensure every evidence item has source_file and line_no.

Acceptance Criteria

- Sample logs produce structured events and signal evidence.
- At least four incident categories are detected reliably.
- Incidents include severity, category, evidence references, and timeline.
- Runtime pipeline never reads ground_truth_eval_only.

Dependencies

- v4 notebook implementation
- sample datasets

Person 6 - Agent Owner B

RCA, remediation, critic, notification, ticket, and cookbook agents

As the incident response team, I want evidence-backed recommendations and action-ready outputs that are useful but safe.

Owned Files / Artifacts

- `src/agents/rca_agent.py`
- `src/agents/remediation_agent.py`
- `src/agents/critic_agent.py`
- `src/agents/notification_agent.py`
- `src/agents/ticket_agent.py`
- `src/agents/cookbook_agent.py`
- `src/reporting/*.py`

Detailed Steps

- 1 Implement RCA agent with probable cause, supporting evidence, missing evidence, confidence, and summary.
- 2 Implement remediation agent with immediate actions, validation checks, rollback notes, and risk level.
- 3 Implement critic agent that flags unsupported claims, missing evidence, and unsafe actions.
- 4 Implement notification agent that returns Slack Block Kit-style JSON and plain text fallback.
- 5 Implement ticket agent that returns JIRA-compatible payloads for P1/P2 only by default.
- 6 Implement cookbook synthesizer that creates detection, triage, remediation, validation, and prevention checklists.
- 7 Add deterministic fallback output for every agent if LLM calls fail.

Acceptance Criteria

- Each agent returns structured Pydantic output.
- RCA and remediation cite evidence and/or retrieved runbook context.
- P1/P2 incidents generate Slack and JIRA payloads; P3 skips JIRA by default.
- Agent failures degrade gracefully and do not crash the run.

Dependencies

- clusters and classifier output
- RAG context
- shared response schema

Person 7 - Integration, QA, Deployment, README, Demo Owner

Submission readiness and end-to-end polish

As the team, we need one owner ensuring the full project runs publicly, is documented, and can be judged quickly.

Owned Files / Artifacts

- `README.md`
- `docs/demo_script.md`
- `docs/screenshots/*`
- `tests/*`
- `requirements.txt`

Detailed Steps

- 1 Own final merge/integration and keep everyone aligned to the shared JSON contract.
- 2 Add README setup instructions for local, hosted, Gradio, FastAPI, and n8n usage.
- 3 Add architecture section using the final uploaded diagram as the source of truth.
- 4 Add scoring section mapping concepts, difficulty, and code quality to judge criteria.
- 5 Create smoke tests for ingest, pipeline, API health, and sample analysis.
- 6 Deploy the public demo and verify from a clean browser session.
- 7 Record a strict under-5-minute demo video and collect screenshots.

Acceptance Criteria

- Public URL works.
- Fresh clone can run deterministic demo.
- README has exact commands and sample flow.
- Submission includes repo, hosted URL, n8n workflow export, and demo video.

Dependencies

- all team outputs
- hosting account
- sample datasets

Final Submission Checklist

- Public hosted Gradio URL works from a clean browser.
- FastAPI /health and /analyze work locally or on hosted backend.
- n8n workflow JSON is committed and importable.
- Sample logs are available and documented.
- README includes setup instructions, architecture, concepts used, and scoring alignment.
- Screenshots show Gradio, n8n, Slack/JIRA preview, and final report.
- Demo video is less than five minutes.
- No hidden ground-truth files are read during runtime analysis.
- Repo contains actual code, not only notebooks or zip files.

Demo Script

- 1 Open the public Gradio app and upload the easy sample incident logs.
- 2 Show the status trace moving through ingest, signals, clustering, agents, RAG, and reports.
- 3 Open the top incident and explain severity, evidence references, timeline, and RCA.
- 4 Show retrieved SOP/runbook context and remediation checklist.
- 5 Show Slack and JIRA previews; then show n8n severity switch routing.
- 6 Download or open the final Markdown/JSON report.
- 7 Close by mapping the project to judging criteria: concepts, difficulty, and code quality.